MATLAB Digital Twin

MATLAB Digital Twin Implementation for Fitsync

Introduction

Fitsync is a revolutionary approach integrating AI, Digital Twin technology, and real-time heart rate monitoring to ensure user safety during workouts. This implementation leverages MATLAB Simulink, an LSTM Autoencoder ML model, and a web interface to provide users with live heart rate tracking, predictions, and anomaly detection.

Digital Twin Implementation in MATLAB Simulink

1. Serial Communication and Data Acquisition

To acquire real-time heart rate (HR) data, we establish a serial connection with an Arduino-based BPM sensor. This is configured using the **Serial Receive** and **Serial Configuration** blocks from the *Simulink Support Package for Arduino Hardware*.

- **Serial Configuration Block**: Configures the baud rate (115200), port (`/dev/cu.usbserial-10`), and other parameters for reliable data transfer.
- **Serial Receive Block**: Collects HR data packets and sends them into Simulink for processing.
- **Convert Block**: Converts the raw input data into a usable format for further computations.

2. Preprocessing and Data Reshaping

Since the LSTM model expects sequential data, we reshape the incoming heart rate values using the **Reshape Input** block. This block normalizes and organizes the input data into a structure compatible with the deep learning model.

3. LSTM-Based Heart Rate Prediction

The ML model used in Fitsync is an **LSTM Autoencoder**, trained on historical heart rate data to detect anomalies. The model is imported into Simulink using the **Deep Learning Model Toolbox** and the **ONNX Model Converter**.

- The trained model is saved as a `.mat` file and loaded into Simulink.
- The **Model Import Block** reads the `.mat` file and integrates it with the Simulink environment.
- The reshaped input is fed into the model, which outputs a predicted heart rate.

4. Denormalization and Anomaly Detection

After prediction, the **Denormalize Heart Rate Block** converts the predicted values back to their original scale for meaningful comparison.

Anomaly detection is implemented by comparing the predicted HR with the actual HR using:

- **Anomaly Detection Block**: If the deviation exceeds a predefined threshold, an anomaly is detected.
- **Threshold Selection**: Based on statistical analysis, thresholds are set dynamically using past HR patterns.

5. Real-Time Communication & Alerts

To ensure real-time data transmission, the system uses the **Control Communication Toolbar** and **UDP Send Blocks**:

- **UDP Send Blocks** transmit both predicted HR and anomaly alerts to the web interface.
- Alerts can be triggered via email/SMS notifications when anomalies are detected.

6. Visualization and User Interface

The **Scope and Display Blocks** provide a visual representation of HR trends and detected anomalies.

- **Web Dashboard Integration**: The **Node.js-based Web Interface** receives UDP packets and displays real-time HR data, predicted HR, and anomaly graphs.

- **MongoDB Database** stores HR logs for historical trend analysis.

Conclusion

The MATLAB Simulink Digital Twin framework enables real-time heart rate monitoring, anomaly detection, and web-based visualization. Future improvements include integrating ECG data and refining anomaly detection thresholds based on user-specific trends.